Follow-along guide — AVD Anywhere

Generated from the demo-guide content by `npm run docs`. Full follow-along: every demo has an attendee version. Azure and the portability pattern need no special hardware; Azure Local and Hybrid steps are hypervisor-agnostic with lab values as variables.

Session: Azure Virtual Desktop on Azure, Azure Local, and AVD Hybrid Platforms. Wed 14 Oct 2026, 13:20 (W. Europe Time).

Replace the placeholders in the commands with your own values:

- `<sub>` — your subscription id
- `<rg>` — your resource group for the AVD control plane
- `<location>` — an Azure region, for example eastus
- `<user>` — a test user, for example user1@yourtenant.onmicrosoft.com (cloud-only works for Azure Files with Entra Kerberos; an on-site AD share such as a SOFS needs a hybrid user synchronized from AD)
- `<storageaccount>` — a globally unique storage account name for the profile share
- `<profile-share>` — the Azure Files share that holds the profile containers
- `<odfc-share>` — the Azure Files share that holds the Office containers

Follow along

One workspace, three host pools, all Available

Goal: See your own workspace, host pools and session host status in one view.

You need

- An AVD workspace with at least one host pool
- Az.DesktopVirtualization module

Steps

1. **List host pools in the workspace's resource group.** List the host pools and the status of their session hosts.

   ```
   Get-AzWvdHostPool -ResourceGroupName <rg> | Select-Object Name, HostPoolType,
   LoadBalancerType
   Get-AzWvdHostPool -ResourceGroupName <rg> | ForEach-Object { Get-AzWvdSessionHost -
   ResourceGroupName <rg> -HostPoolName $_.Name | Select-Object Name, Status, Session }
   ```

   - You should see: Each host pool lists its session hosts with Status Available.

Troubleshooting

- Status Unavailable: check the host's network access to the AVD service URLs and the agent version.

Clean up

- None; read-only.

In the repo: `automation/demo/avd/scripts/Show-AvdState.ps1`

Identity: three paths, one proof

Goal: Enable Entra Kerberos on an Azure Files share and confirm a Windows 11 Entra-joined VM obtains a ticket.

You need

- An Entra-joined Windows 11 VM
- A storage account (FileStorage or StorageV2) with a file share
- Rights to grant admin consent
- Note: cloud-only identities are supported (GA) for Azure Files with Entra Kerberos; check the current Windows 11 build requirement on Microsoft Learn

Steps

1. **Enable Entra Kerberos on the storage account.** In the portal: Storage account > File shares > Identity-based access > Microsoft Entra Kerberos > Set up. Enable it, then grant admin consent to the generated app registration.
    - You should see: An app named [Storage Account] .file.core.windows.net exists with consent granted.
2. **Allow the ticket to be issued without MFA.** Exclude the storage account's app from Conditional Access policies that require MFA; the ticket request is silent and cannot prompt.
    - You should see: The app appears under Exclude in the policy.
3. **Enable cloud-only group support.** In the app registration manifest add the tag kdc_enable_cloud_group_sids. Use security groups that are not nested.
    - You should see: The tag appears in the manifest tags array.
4. **Enable ticket retrieval on the client.** Set the registry value that lets the client request the Entra Kerberos ticket and restart the client.

    ```
    reg add HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters /v
    CloudKerberosTicketRetrievalEnabled /t REG_DWORD /d 1 /f
    ```

    - You should see: Value is 1.
5. **Assign share-level permission.** Assign Storage File Data SMB Share Contributor to your users group on the share.

    ```
    New-AzRoleAssignment -ObjectId <group object id> -RoleDefinitionName 'Storage File Data
    SMB Share Contributor' -Scope <share resource id>
    ```

    - You should see: The role assignment lists on the share.
6. **Request the ticket.** Restart the client, then sign in again to the VM as an Entra user (the registry value takes effect at sign-in) and request a ticket for the storage account.

```
klist get cifs/<storageaccount>.file.core.windows.net
```

- You should see: klist shows a ticket for the cifs service of the storage account.

Troubleshooting

- No ticket: check the client is Entra-joined or hybrid-joined, the registry value, and the MFA exclusion.
- Access denied after the ticket: check the share-level role and the NTFS ACL.

Clean up

- Remove the role assignment and disable Entra Kerberos on the storage account if it was a test.

In the repo: `automation/landing-zones/avd/scripts/Set-AvdStorageEntraKerberos.ps1`, `handouts/Identity_Reference_Architecture.md`

AVD on Azure: host pool and session hosts

Goal: Create a pooled host pool, desktop application group and workspace.

You need

- Az.DesktopVirtualization
- Microsoft.DesktopVirtualization provider registered
- A resource group

Steps

1. **Create the host pool.** Create a pooled, breadth-first host pool with a session limit.

   ```
   $hp = New-AzWvdHostPool -ResourceGroupName <rg> -Name hp-demo-azure -Location <location> -
   HostPoolType Pooled -LoadBalancerType BreadthFirst -PreferredAppGroupType Desktop -
   MaxSessionLimit 4
   ```

   - You should see: Get-AzWvdHostPool returns hp-demo-azure.
2. **Create the desktop application group.** Create a desktop application group for the pool.

   ```
   $ag = New-AzWvdApplicationGroup -ResourceGroupName <rg> -Name ag-demo-azure -Location
   <location> -HostPoolArmPath $hp.Id -ApplicationGroupType Desktop
   ```

   - You should see: The application group lists the host pool.
3. **Create the workspace and register the group.** Create the workspace and add the application group.

   ```
   New-AzWvdWorkspace -ResourceGroupName <rg> -Name ws-demo -Location <location> -
   ApplicationGroupReference $ag.Id
   ```

   - You should see: The workspace lists the application group.
4. **Entitle a user group.** Assign the Desktop Virtualization User role on the application group.

```
New-AzRoleAssignment -ObjectId <group object id> -RoleDefinitionName 'Desktop
Virtualization User' -Scope $ag.Id
```

- You should see: The users appear on the application group's assignments.

Troubleshooting

- Provider not registered: Register-AzResourceProvider -ProviderNamespace Microsoft.DesktopVirtualization.

Clean up

- Remove the workspace, application group and host pool when finished.

In the repo: `automation/avd/control-plane`

One image artifact: Image Builder to gallery to host pool

Goal: Build a Windows 11 multi-session image with Azure Image Builder into a Compute Gallery.

You need

- Compute Gallery with an image definition
- Image Builder identity with the gallery roles
- Microsoft.VirtualMachineImages provider registered

Steps

1. **Resolve an exact source image version.** Pin an exact marketplace version for reproducibility (Image Builder also accepts latest, resolved at build time).

   ```
   ./automation/avd/images/azure/scripts/Get-PlatformImageVersion.ps1 -Location <location> -
   Publisher MicrosoftWindowsDesktop -Offer office-365 -Sku win11-25h2-avd-m365
   ```

   - You should see: An exact version number is returned.
2. **Deploy the template.** Deploy the Image Builder template from the repo with your gallery and identity.

   ```
   az deployment group create -g <rg> --template-file
   automation/avd/images/azure/bicep/main.bicep --parameters <your parameter file>
   ```

   - You should see: The template shows provisioning Succeeded.
3. **Start the build.** Start the build; it takes 45 to 90 minutes.

   ```
   ./automation/avd/images/azure/scripts/Start-ImageBuild.ps1 -SubscriptionId <sub> -
   ResourceGroupName <rg> -Execute
   ```

   - You should see: Run state moves to Running, then Succeeded.
4. **Confirm the gallery image version.** List the versions of the image definition.

```
Get-AzGalleryImageVersion -ResourceGroupName <rg> -GalleryName <gallery> -
GalleryImageDefinitionName <definition> | Select-Object Name, ProvisioningState
```

    - You should see: A new version with ProvisioningState Succeeded.

Troubleshooting

- Build fails at the customiser: open the staging resource group's customization log.

Clean up

- Delete the image template and old image versions to avoid storage cost.

In the repo: `automation/avd/images/azure`

AVD Insights

Goal: Enable AVD Insights for a host pool.

You need

- A Log Analytics workspace
- Session hosts with Azure Monitor Agent

Steps

1. **Open Insights and run the configuration workbook.** In the portal open Azure Virtual Desktop > Insights, select the subscription, and use Open configuration workbook.
    - You should see: The configuration workbook reports diagnostics enabled for the host pool and workspace.
2. **Associate the Insights data collection rule with the hosts.** Create the data collection rule from the workbook and associate it with the session hosts.
    - You should see: Host performance counters and events appear after the first collection interval.

Troubleshooting

- No data: allow time for the first collection; confirm the Azure Monitor Agent is installed.

Clean up

- Delete the data collection rule and association if it was a test.

In the repo: `handouts/Deployment_Checklist.md`

Data collection rules that keep costs sane

Goal: Measure what your session hosts cost to monitor and keep only what you need.

You need

- A Log Analytics workspace with host data

Steps

1. **Measure ingestion by table.** Run the query in your workspace.

   ```
   Usage | where TimeGenerated > ago(30d) | where IsBillable == true | summarize GB = sum(Quantity)/1000.0 by DataType | order by GB desc
   ```

   - You should see: A ranked list of tables by billable GB.
2. **Review the DCR data sources.** Open the data collection rule and review counters, sampling rates and event channels; remove what you do not use.
   - You should see: Only the counters and channels that Insights and your alerts need remain.

Clean up

- None.

In the repo: `automation/landing-zones/avd`

Sizing AVD on Azure Local: the two public guides and this lab's numbers

Goal: Size an AVD solution on Azure Local bottom-up for your own users: session hosts and the SOFS that serves the FSLogix profiles.

You need

- A browser: https://azurelocal.cloud/azurelocal-avd/ and https://azurelocal.cloud/azurelocal-sofs-fslogix/architecture/capacity-planning/
- Your user count, profile size per user type and growth estimate
- Your cluster's node count, capacity drives per node and memory per node

Steps

1. **Read the AVD on Azure Local architecture.** Open https://azurelocal.cloud/azurelocal-avd/ and read the Architecture and Deployment Scenarios pages: session hosts are Azure Local VMs on a logical network, managed from Azure. Pick the scenario closest to yours and size the session hosts from user density (vCPU and memory per user).
   - You should see: You have a session-host count and size (vCPU, memory, OS disk) for your users.
2. **Size the profile storage.** Open https://azurelocal.cloud/azurelocal-sofs-fslogix/architecture/capacity-planning/ and compare https://azurelocal.cloud/azurelocal-sofs-fslogix/architecture/scenarios/. Compute usable = users × profile size × (1 + growth); guest pool = usable × guest mirror copies; data disks per VM = pool ÷ SOFS VMs; host volume per VM = OS disk + data disks + overhead; raw = host volumes × host mirror copies. Worked example from the session (two hosts): 20 users × 30 GB + 20 % = 720 GB usable → 1.44 TB guest two-way pool → 2 VMs × 4 × 256 GB (one disk per VM kept as repair reserve) → two 1.5 TiB host volumes → 6 TiB raw. On two hosts use two SOFS VMs, one per host, with a cloud witness: a third VM would share a host and losing that host would take the guest quorum.
   - You should see: You have usable, guest pool, data-disk, host-volume and raw figures for the profiles.
3. **Check the fit on the cluster.** Add the session-host and other VM volumes to the profile volumes, keep one capacity drive per node unallocated as the repair reserve, and check that every VM can restart on the remaining nodes when one node is down (vCPU and memory).

    - You should see: Allocated volumes fit inside usable capacity minus the reserve, and the VMs fit on N-1 nodes.

Troubleshooting

- Raw-to-usable looks far worse than 4:1 at small scale: the fixed OS disks and the second mirror dominate small designs; compare the guide's small-shop scenario.
- Does not fit on N-1 nodes: reduce density per host, add a node, or accept that some VMs stay down during a node outage, and write that down.

Clean up

- None: nothing is deployed.

In the repo: `handouts/Realm_Comparison.md`

Plain Hyper-V VMs seen in Azure only as Arc machines

Goal: Make an ordinary Windows VM on any hypervisor an Arc-enabled server.

You need

- A Windows 11 Enterprise VM (any hypervisor)
- Rights to onboard Arc machines (Azure Connected Machine Onboarding on the resource group)
- Outbound access to the Arc endpoints

Steps

1. **Create the onboarding principal.** Create a service principal scoped only to the Arc resource group.

```
$sp = New-AzADServicePrincipal -DisplayName spn-arc-onboard
New-AzRoleAssignment -ObjectId $sp.Id -RoleDefinitionName 'Azure Connected Machine
Onboarding' -Scope /subscriptions/<sub>/resourceGroups/<arc rg>
```

    - You should see: The role assignment exists on the Arc resource group.
2. **Install and connect the Connected Machine agent in the guest.** In the guest, in an elevated session, download and run AzureConnectedMachineAgent.msi from the Microsoft download link in the Azure Arc documentation, then run the onboarding. Pass the secret in memory and never store it.

```
azcmagent connect --service-principal-id <app id> --service-principal-secret <secret> --
tenant-id <tenant> --subscription-id <sub> --resource-group <arc rg> --location <location>
--resource-name <vm name>
```

    - You should see: azcmagent show reports Connected.

Troubleshooting

- Agent cannot connect: check outbound access to the Arc endpoints and the system clock.

Clean up

- azcmagent disconnect, then delete the Arc machine resource.

In the repo: `automation/avd/session-hosts-hybrid/scripts/Install-ArcAgent.ps1`

The AVD Hybrid extension on an Arc machine

Goal: Register an Arc-enabled Windows 11 machine as an AVD Hybrid session host.

You need

- A standard host pool created without VMs and with a managed identity
- Reader for that identity on the Arc resource group
- An Arc-enabled, Entra-joined Windows 11 Enterprise VM
- Az.DesktopVirtualization and Az.ConnectedMachine

Steps

1. **Generate a registration token.** Create a short-lived token; keep it in memory only.

   ```
   $expires = (Get-Date).ToUniversalTime().AddHours(2).ToString('yyyy-MM-
   ddTHH:mm:ss.fffffffZ')
   $token = (New-AzWvdRegistrationInfo -ResourceGroupName <rg> -HostPoolName <pool> -
   ExpirationTime $expires).Token
   ```

   - You should see: $token is populated.
2. **Install the extension.** Install the AVD Hybrid extension on the Arc machine with the token as a protected setting.

   ```
   New-AzConnectedMachineExtension -Name 'Microsoft.AzureVirtualDesktop.CloudDeviceExtension'
   -ResourceGroupName <arc rg> -MachineName <machine> -Location <location> -Publisher
   'Microsoft.AzureVirtualDesktop' -ExtensionType 'CloudDeviceExtension' -ProtectedSetting @{
   registrationToken = $token }
   ```

   - You should see: The extension shows Succeeded.
3. **Wait for the host.** The host can take up to 15 minutes to become Available after the extension succeeds.

   ```
   Get-AzWvdSessionHost -ResourceGroupName <rg> -HostPoolName <pool> | Select-Object Name,
   Status
   ```

   - You should see: The host shows Available.

Troubleshooting

- Host stays Unavailable: confirm Reader for the pool's managed identity on the Arc resource group and outbound access to the AVD service URLs.

Clean up

- Remove the extension and the session host from the pool.

In the repo: `automation/avd/session-hosts-hybrid/scripts/Register-HybridHosts.ps1`

The profile share, per-user containers and the settings that make them portable

Goal: Configure FSLogix profile and ODFC containers on an Azure Files share and confirm a container is created.

You need

- A session host (Entra-joined)
- A share with Entra Kerberos enabled (see the identity demo)
- FSLogix installed on the host

Steps

1. **Apply the FSLogix settings.** Apply the repo's configuration profile (replace the share path with yours).

   ```
   ./automation/avd/fslogix/scripts/Set-FslogixHostConfig.ps1 -ProfileShareUnc \\
   <storageaccount>.file.core.windows.net\<profile-share> -OdfcShareUnc \\
   <storageaccount>.file.core.windows.net\<odfc-share> -Execute
   ```

   - You should see: Test-FslogixHostConfig reports the settings applied.
2. **Sign in once and check the container.** Sign in to the host as a test user, sign out, then list the share.

   ```
   Get-ChildItem \\<storageaccount>.file.core.windows.net\<profile-share> -Recurse -Filter
   *.vhdx
   ```

   - You should see: A Profile VHDX (and ODFC VHDX) exists under the user's folder.
3. **Publish redirections.xml.** Publish the exclusion file to the share so every host uses the same one.

   ```
   ./automation/avd/fslogix/scripts/Publish-FslogixRedirections.ps1 -ProfileShareUnc \\
   <storageaccount>.file.core.windows.net\<profile-share> -Execute
   ```

   - You should see: redirections.xml is in the redirections folder of the share.

Troubleshooting

- Temporary profile on sign-in: check the share path, the user's role on the share and the ticket (klist).

Clean up

- Delete the test user's folder from the share.

In the repo: `automation/avd/fslogix`

One repo, one config, three realms

Goal: Preview a change to the AVD control plane without applying it.

You need

- The repo
- An existing control-plane deployment or a test resource group

Steps

1. **Run a what-if.** Change one parameter (for example the host pool session limit) and run what-if.

   ```
   az deployment group what-if -g <rg> --template-file automation/avd/control-
   plane/bicep/main.bicep --parameters <your parameter file>
   ```

   - You should see: The output lists one modified property.

Troubleshooting

- What-if shows noise: some properties are service-computed; compare only the properties you changed.

Clean up

- None; what-if creates nothing.

In the repo: `automation/avd/control-plane`

Crossing 1: three host pools, one workspace

Goal: Check your host pools are healthy before testing portability.

You need

- Two host pools, one user group per pool

Steps

1. **Check host status.** List session hosts and sessions.

   ```
   Get-AzWvdHostPool -ResourceGroupName <rg> | ForEach-Object { Get-AzWvdSessionHost -
   ResourceGroupName <rg> -HostPoolName $_.Name | Select-Object Name, Status, Session }
   ```

   - You should see: All hosts Available with no sessions.

Clean up

- None.

In the repo: `automation/demo/avd/scripts/Show-AvdState.ps1`

Crossing 2: the user is entitled to the right realm

Goal: Control which desktop a user sees with group membership.

You need

- Two application groups, each assigned to a different Entra group
- A test user

Steps

1. **Add the user to the first group.** Add the test user to the group assigned to the first application group.

```
Add-AzADGroupMember -TargetGroupObjectId <group A id> -MemberUserPrincipalName <user>
```

   - You should see: The user sees one desktop in Windows App.

2. **Move the user to the second group.** Remove from group A and add to group B; refresh the feed.

```
Remove-AzADGroupMember -GroupObjectId <group A id> -MemberUserPrincipalName <user>
Add-AzADGroupMember -TargetGroupObjectId <group B id> -MemberUserPrincipalName <user>
```

   - You should see: After refresh the feed shows the second desktop only.

Troubleshooting

- Feed unchanged: allow a few minutes, then refresh or sign out and in of Windows App.

Clean up

- Restore the original group membership.

In the repo: `automation/demo/avd/scripts/Switch-UserHostPool.ps1`

Crossing 3: profile portability across realms

Goal: Prove a profile follows a user across two host pools.

You need

- Two host pools using the same FSLogix share and configuration
- A test user entitled to both pools (one at a time)

Steps

1. **Create state in the profile.** Sign in to pool 1, create a file on the desktop and change a setting such as the wallpaper.
   - You should see: File and setting present.
2. **Sign out and confirm the container released.** Sign out; confirm no open handle on the container before signing in elsewhere.

```
./automation/demo/avd/scripts/Test-ProfilePortability.ps1 -UserPrincipalName <user>
```

   - You should see: The container shows a recent last write and no open handles.
3. **Sign in to pool 2.** Sign in to the second pool.

```
hostname
```

   - You should see: The hostname belongs to pool 2 and the file and setting are present.

Troubleshooting

- File missing: confirm OneDrive Known Folder Move is off so you are testing FSLogix, and that both pools use the same share path.

Clean up

- Delete the test file and restore the setting.

In the repo: `automation/demo/avd/scripts/Test-ProfilePortability.ps1`, `automation/avd/fslogix`

Crossing 4: forced host failure on Hybrid

Goal: See what a host failure does to a session and what survives.

You need

- A host pool with two Available hosts (VMs you can stop)
- FSLogix profile on shared storage
- A test user with a session

Steps

1. **Start a session and leave an unsaved document open.** Sign in and open a document without saving it.
    - You should see: The session is on host 1.
2. **Stop host 1 hard.** Stop the VM without a graceful shutdown (for a Hyper-V VM: Stop-VM -TurnOff).

    ```
    Stop-VM -Name <host 1> -TurnOff
    ```

    - You should see: The host shows Unavailable in the pool after a minute or two.
3. **Open a new session.** Reconnect from Windows App; the broker sends you to host 2.

    ```
    hostname
    ```

    - You should see: You are on host 2; your saved files and settings are present; the unsaved document is not.
4. **Restart host 1.** Start the VM; AVD does not do this for you in a Hybrid realm.

    ```
    Start-VM -Name <host 1>
    ```

    - You should see: The host returns to Available.

Troubleshooting

- Profile will not attach: a stale handle on the container; wait for the SMB session to expire or close the handle on the share.

Clean up

- Ensure both hosts are Available.

In the repo: `automation/demo/avd/scripts/Stop-SessionHost.ps1`, `automation/demo/avd/scripts/Restore-SessionHost.ps1`

Crossing 5: one monitoring layer

Goal: Query all your hosts from one workspace.

You need

- Azure Monitor Agent on hosts reporting to one workspace

Steps

1. **Run a heartbeat query.** Show the last heartbeat per host.

   ```
   Heartbeat | where TimeGenerated > ago(1h) | summarize last = max(TimeGenerated) by
   Computer | order by Computer asc
   ```

   - You should see: Every host you expect, with a recent last heartbeat.

Troubleshooting

- Missing host: check the Azure Monitor Agent and the DCR association.

Clean up

- None.

In the repo: `handouts/Deployment_Checklist.md`

Crossing 6: the Observability Agent explains the failure (preview)

Goal: Investigate an Azure Monitor alert with the Azure Copilot Observability Agent (preview) and keep the result as an issue.

You need

- Access to Azure Copilot in your tenant
- An Azure Monitor workspace associated with the subscription (it holds the issues)
- Issue Contributor, Monitoring Contributor or Contributor on that workspace
- A fired Azure Monitor alert, for example a session-host alert, in a region where the agent is available

Steps

1. **Start an investigation from an alert.** Open a fired alert in Azure Monitor > Alerts and select Investigate. Confirm the scope the agent proposes.
   - You should see: The agent reports its plan, then findings, hypotheses ruled out and next steps.
2. **Keep the result.** Select Create issue to save the investigation as an Azure Monitor issue in your Azure Monitor workspace.
   - You should see: The issue is listed under Monitor > Issues with the investigation attached.

Troubleshooting

- Asked for an Azure Monitor workspace: associate the subscription with one (once per subscription).
- No Investigate action: check that your account has Azure Copilot access.

Clean up

- Close or delete test issues; chat and investigations are billed per use.

In the repo: `handouts/Deployment_Checklist.md`

Read along

The foundation as code

Goal: Understand how the foundation is organised so you can adapt it.

You need

- The repo checked out

Steps

1. **Read the solution layout.** Open automation/landing-zones/avd and compare the Bicep and Terraform folders; both take the inputs listed in solution.yml.

   ```
   Get-ChildItem automation/landing-zones/avd -Recurse -Depth 2 | Select-Object FullName
   ```

   - You should see: You can find bicep/, terraform/ and solution.yml.
2. **Run a what-if of the foundation against your subscription.** Copy the example parameter file, replace the values with yours, and run a what-if. Nothing is created.

   ```
   az deployment sub what-if --location <location> --template-file automation/landing-
   zones/avd/bicep/main.bicep --parameters <your parameter file>
   ```

   - You should see: A list of resources that would be created; no errors.

Troubleshooting

- Provider not registered: register Microsoft.DesktopVirtualization first.

Clean up

- None; what-if creates nothing.

In the repo: `automation/landing-zones/avd/README.md`, `automation/landing-zones/avd/solution.yml`

AVD on Azure Local: host pool and a session host as an Azure Local VM

Goal: Understand how an Azure Local VM becomes an AVD session host; the steps need an Azure Local cluster.

You need

- An Azure Local cluster with an Arc Resource Bridge and a custom location (not required to read)

Steps

1. **Create the VM from Azure.** Create an Azure Local VM from a gallery image on a logical network, then register it to the host pool with a registration token passed in memory.

   ```
   az stack-hci-vm create --name <vm> --resource-group <rg> --custom-location <custom
   location id> --image <image name> --admin-username <admin> --admin-password <read from a
   ```

```
secure prompt, never typed into a script> --nics <nic name> --storage-path-id <storage
path id>
```

    - You should see: The VM lists as Succeeded under Azure Local virtual machines.

2. **Register to the host pool.** Run the registration step with a token generated at deploy time.

```
./automation/avd/session-hosts-azure-local/scripts/Register-AvdSessionHostsAzureLocal.ps1
-SubscriptionId <sub> -ResourceGroupName <vm rg> -HostPoolResourceGroup <rg> -HostPoolName
<pool> -TokenScriptPath ./automation/avd/control-plane/scripts/New-
AvdRegistrationToken.ps1 -Execute
```

    - You should see: The host appears Available in the pool.

Troubleshooting

- Host Unavailable: check outbound access to the AVD service URLs from the logical network.

Clean up

- Delete the VM and remove the session host from the pool.

In the repo: `automation/avd/session-hosts-azure-local`

GPU on Azure Local: one L4 by DDA on a dedicated session host

Goal: Understand GPU by DDA versus GPU partitioning on Azure Local and what each needs.

In the repo: `automation/azure-local/cluster-configure/gpu`